

Programming 2025/2026

Laboratory

Session 4 (April 24)

Tuples (and Functions)

First Cycle Degree/Bachelor in Genomics
Dept. Of Pharmacy and Biotechnology (FaBiT)
University of Bologna



Tuples

- A tuple is an **immutable sequence** of heterogeneous elements separated by “,”

```
data = tuple ()  
data = ('Mario', 'Rossi', 'M', 20)
```

- A tuple with a single element: `data = ('Mario',)`
- Operations that can be performed on tuples:
 - **+** concatenate two tuples
 - ***** repeat the elements in the tuple (`t = (1,2)*5?`)
 - selection operator `[]` and the slicing operators `[:]`, `:::`
 - **in** and **not in** operators
 - **len()**

Tuples

- We can unpack a tuple. E.g.,

```
(firstname, lastname, gender, age) = data  
fn, ln, g, a = data
```

```
>>> data = ("Mario", "Rossi", "M", 20)  
>>> data  
( 'Mario', 'Rossi', 'M', 20)  
>>> (firstname, lastname, gender,age)=data  
>>> fn, ln, g,a=data  
>>> print(firstname, lastname, gender,age)  
Mario Rossi M 20  
>>> print(fn, ln, g,a)  
Mario Rossi M 20
```

Example

```
def time(seconds):  
    h = seconds // 3600  
    s_remainder = seconds % 3600  
    m = s_remainder // 60  
    seconds = s_remainder % 60  
    print(h, m, seconds)
```

Example

```
def time(seconds):  
    h = seconds // 3600  
    s_remainder = seconds % 3600  
    m = s_remainder // 60  
    seconds = s_remainder % 60  
    print(h, m, seconds)
```

```
def time(seconds):  
    h = seconds // 3600  
    s_remainder = seconds % 3600  
    m = s_remainder // 60  
    seconds = s_remainder % 60  
    return h, m, seconds # similarly return (h, m, seconds)
```

```
hs, ms, ss = time(10000) # unpacking the tuple returned by the function call
```

Exercise 7.1

- Write two functions, where given a tuple of integers **t** and another integer **n**:
 - the first function returns a new tuple in which **n** is inserted to the head (first element) of **t**,
 - the second function returns a new tuple in which the head (first element) of **t** is replaced by **n**.

No iteration allowed.

Exercise 7.2

- Write a function that given two tuples **t1** and **t2** and a non-negative integer **n**, returns a new tuple obtained by inserting all the elements **t2** at position **n** of **t1**.
No iteration allowed.

E.g., given $t1 = (1, 3, 2)$, $t2 = (9, 7)$, $n = 1$, the function returns $(1, 9, 7, 3, 2)$.

Exercise 7.3

- Write a function that given a tuple of numbers, which represents grades, returns the average grade. Ask the user to insert the grades as a tuple and parse the input with **eval()**.

Exercise 7.4

- Write a function that prints the Floyd's Triangle of size **n**, which is passed as a parameter. The *i*-th row of the triangle is a tuple of numbers.

E.g., the triangle with **n** = 5 is:

```
1  (1 , )
2  (2 , 3)
3  (4 , 5 , 6)
4  (7 , 8 , 9 , 10)
5  (11 , 12 , 13 , 14 , 15)
```

- Tip: the first element of each row corresponds to the number of the previous row (starting to count rows from 1) added to its first element.

Exercise 7.5

- Write a function that given a tuple **t** behaves as follows:
 - returns **None** if **t** is not a tuple,
 - verifies that **t** contains only integers (using a while loop), otherwise returns **False**,
 - returns **True** if in **t** each element (except the first) is greater than the sum of the previous elements (using a while loop), otherwise returns **False**.

Exercise 7.6

- Write a program that stores student grades in a tuple and shows a menu of choices to the user:

- 1) insert a new grade in the tuple

- 2) view the marks stored in the tuple

- 3) print the average of the grades stored in the tuple,

- 0) exit the program

and performs the actions related to the choice, repeatedly, until the user enters 0.

- Tip: for 3, use the function defined in exercise 7.3.